

CS6004NT

Application Development

Previously

On the previous week, we covered the following topics:

- Introduction to .NET and C#
 - Module Overview
 - .NET Overview and Ecosystem
 - Key Features of C# and .NET
 - Required Software (SDK, IDE, Workloads)
 - Working with Visual Studio, Rider, and the Command-Line Interface (CLI)

Overview of this week

This week, we will be covering following topics:

- C# Syntax Essentials
- Variables & Memory
- Strings
- Arrays
- Generic Collections

C# Syntax Essentials

1. C# Syntax Essentials

1. Comments, Indentation, Statements, Blocks, Expressions
2. Top-Level Statements
3. Identifiers and Naming Conventions

2. Variables & Memory

1. Declaring & Initializing Variables
2. const & readonly
3. Built-in Types: Guid, DateTime, TimeSpan

3. Strings

1. String literals
2. Interpolated Strings

4. Arrays

1. Single-Dimensional Arrays
2. Common Array Methods

5. Generic Collections

1. List
2. Dictionary

C# Syntax Essentials

Comments

- **Comments** in programming are used to **explain code**, **make it more readable**, or **temporarily disable code** during debugging.
- Comments are **ignored** by **the compiler**.

Single-Line Comments

- **Purpose:** Short or one line comment to a code
- **Syntax:** // Comment text

```
//calculate the sum of two numbers  
int result = 5 + 3;
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

C# Syntax Essentials

Comments

Multi - Line Comments

- Purpose:** Group related explanations or disable code blocks.
- Syntax:** `/* Comment text */`

```
/*  
 * This function adds two integers and returns the result.  
 * Parameters:  
 * a - First number  
 * b - Second number  
 */  
0 references  
int Add(int a, int b)  
{  
    return a + b;  
}
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

C# Syntax Essentials

Comments

Document Comments

•**Purpose:** special comments used to generate documentation for your code.

•**Syntax:** They are written using triple slashes (///) and placed **immediately before** a class, method, property, or other member.

```
1  /// <summary>
2  /// Adds two integer numbers and returns their sum.
3  /// </summary>
4  /// <param name="a">The first integer to add.</param>
5  /// <param name="b">The second integer to add.</param>
6  /// <returns>The sum of <paramref name="a"/> and <paramref name="b"/>.
   /// </returns>
7  public int Add(int a, int b)
8  {
9      return a + b;
10 }
11
```

1. C# Syntax Essentials

1. Comments, Indentation, Statements, Blocks, Expressions

2. Top-Level Statements

3. Identifiers and Naming Conventions

2. Variables & Memory

1. Declaring & Initializing Variables

2. const & readonly

3. Built-in Types: Guid, DateTime, TimeSpan

3. Strings

1. String literals

2. Interpolated Strings

4. Arrays

1. Single-Dimensional Arrays

2. Common Array Methods

5. Generic Collections

1. List

2. Dictionary

C# Syntax Essentials

Indentations

Indentation

- **Indentation** in programming means **adding spaces or tabs at the beginning of lines of code** to visually structure the program
- **Why It Matters:** Improves **readability**, even though C# doesn't enforce it syntactically.
- **Best Practice:** 4 spaces or 1 tab per level.

```
if(age >= 18)
{
    //Proper indentation for nested blocks
    Console.WriteLine("You can vote!");
}
else
{
    Console.WriteLine("Come back in a few years.");
}
```



1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

C# Syntax Essentials

Statements & Blocks

Statements

- Actions that execute code (end with ;).
 - **Simple Statement:** `int x = 10;`
 - **Compound Statement:** A block of code inside {}.

Blocks

- Group multiple statements (e.g., **loops**, **conditionals**).

```
// Block inside a loop
for(int i =0; i<5; i++)
{
    Console.WriteLine(i); // Nested statement
}
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

C# Syntax Essentials

Expressions

Expressions

- Combinations of values, variables, and operators that evaluate to a result.

```
// Arithmetic Expression
int num = 5 + 3;

// Logical Expression
Console.WriteLine(name == "John");

// Object Creation Expression
List<int> myList = new List<int>();
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

C# Syntax Essentials

Combined Example

```
// Single-line comment
int score = 90;

/*
 * Multi-line comment:
 * Check if the score is passing */
if(score >= 60) { // Statement with block
    // Expression: Calculate grade
    string grade = score >= 90 ? "A" : "B" ;
    Console.WriteLine($"Grade: {grade}");
}
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

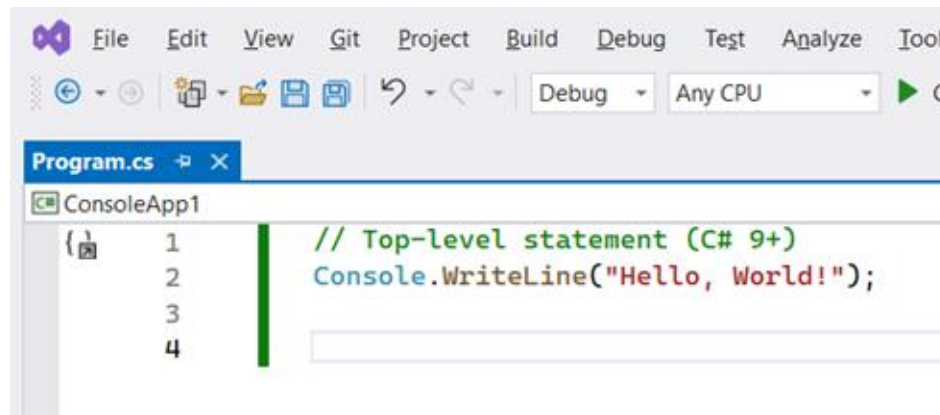
C# Syntax Essentials

Top-Level Statements

Top Level Statements

- Code placed directly in a file without being nested inside a class or method.
- Introduced in C# 9.0 (.NET 5+), simplifies entry points and utility files.

```
// Traditional style (pre-C# 9)
0 references
class Program
{
    0 references
    static void Main()
    {
        Console.WriteLine("Hello, World!");
    }
}
```



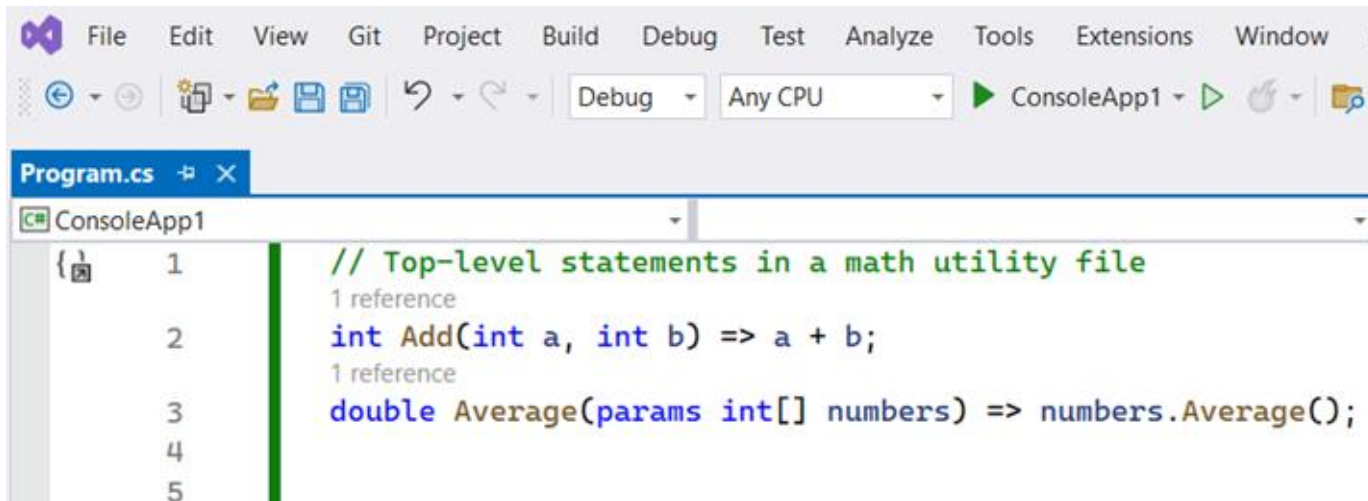
1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
1. Arrays
 2. Single-Dimensional Arrays
 3. Common Array Methods
2. Generic Collections
 2. List
 3. Dictionary

C# Syntax Essentials

Top-Level Statements

When to use them

- Simple console apps.
- Utility scripts (e.g., data processing).
- Reducing boilerplate code.



The screenshot shows the Visual Studio IDE with a C# console application named 'ConsoleApp1'. The code in 'Program.cs' is as follows:

```
1 // Top-level statements in a math utility file
2 int Add(int a, int b) => a + b;
3 double Average(params int[] numbers) => numbers.Average();
4
5
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

C# Syntax Essentials

Top-Level Statements

Restrictions on top – level statements

- Access modifiers (like public, private) **cannot be used**.
- You cannot have top-level statements spread across **multiple .cs files** in the same project.

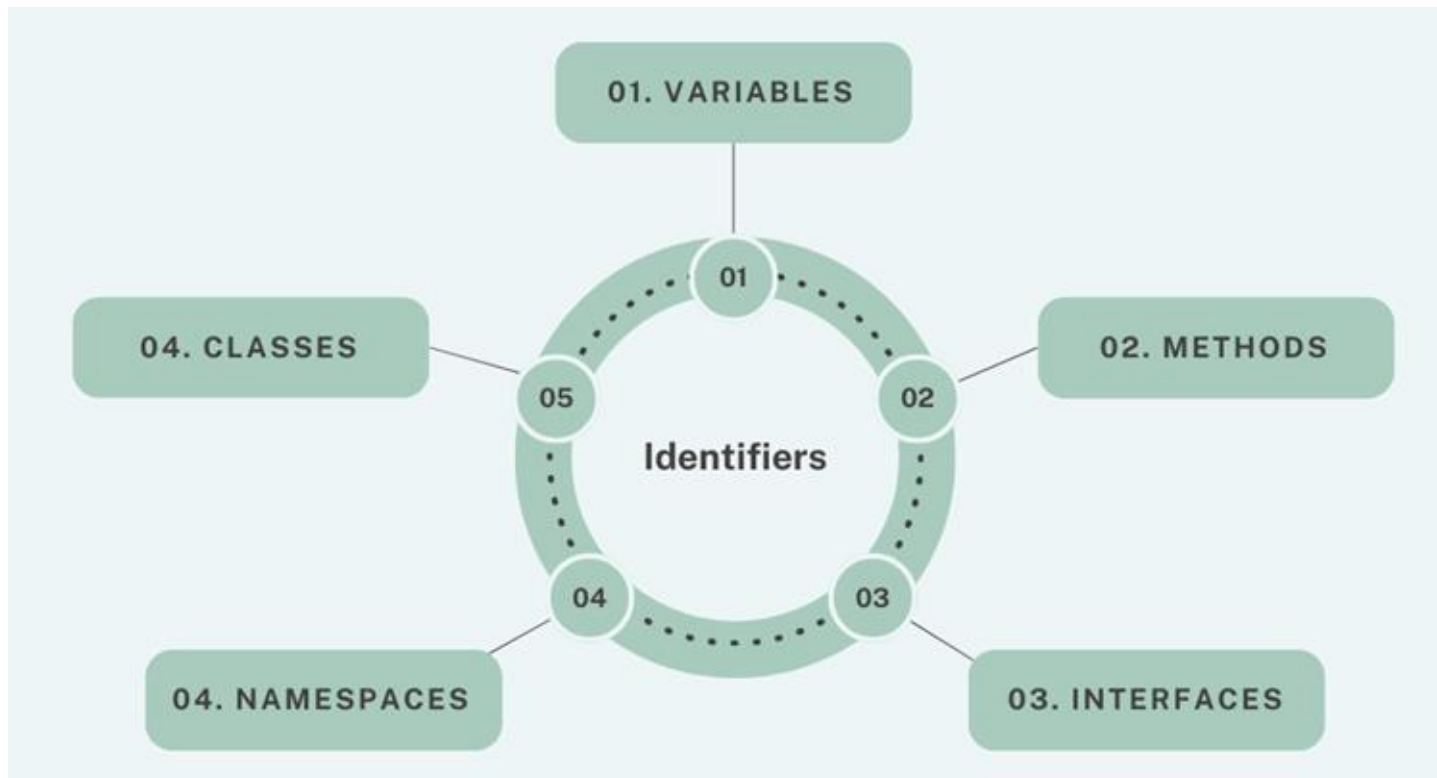
```
1
2 // Invalid: Can't use 'private' at top level
3 private void Log(string message)
4 {
5     /* .. */
6 }
7
8 Log("Hello, World!");
9
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

C# Syntax Essentials

Identifiers and Naming Conventions

Identifiers are names used to identify **variables, methods, classes, namespaces, interfaces** and other program elements.



1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

C# Syntax Essentials

Identifiers and Naming Conventions

Rules for naming identifiers:

- Must start with a **letter** or **underscore**. **Cannot** start with **digits**. Valid example: `_name`, `location`
- Can include **letters**, **digits (0-9)**, or **underscores**. Valid example: `first_name1`
- **Avoid reserved** keywords (e.g., `int`, `for`, etc.)
- Are **Case Sensitive** meaning `public` and `Public` are two different words.

```
//Invalid identifiers  
  
string 1variable; // starts with a digit  
  
int my-variable; // contains a hyphen  
  
double public; // Reserved keywords
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. `const` & `readonly`
 3. Built-in Types: `Guid`, `DateTime`, `TimeSpan`
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

C# Syntax Essentials

Identifiers and Naming Conventions

Naming Conventions for identifiers

Style	Example	Use Case
Pascal Case	CalculateTotal	Methods, Classes
camelCase	userName	Variables, parameters

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

C# Syntax Essentials

Why It Matters

Naming Conventions Benefits

- Readability.
- Maintainability.
- Avoid confusion (e.g., UserName vs. username).

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Arrays

C# Syntax Essentials

Quiz (Naming Convention)

Which is the correct naming convention for a constant?

- a.) maxUsers
- b.) MAX_USERS
- c.) MaxUsers
- d.) maximumUsers



- 1. C# Syntax Essentials
 - 1. Comments, Indentation, Statements, Blocks, Expressions
 - 2. Top-Level Statements
 - 3. Identifiers and Naming Conventions
- 2. Variables & Memory
 - 1. Declaring & Initializing Variables
 - 2. const & readonly
 - 3. Built-in Types: Guid, DateTime, TimeSpan
- 3. Strings
 - 1. String literals
 - 2. Interpolated Strings
- 4. Arrays
 - 1. Single-Dimensional Arrays
 - 2. Common Array Methods
- 5. Generic Collections
 - 1. List
 - 2. Dictionary

C# Syntax Essentials

Quiz (Naming Convention)

Which is the correct naming convention for a constant?

- a.) maxUsers ❌
- b.) MAX_USERS ❌
- c.) **MaxUsers** ✅
- d.) maximumUsers ❌



1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

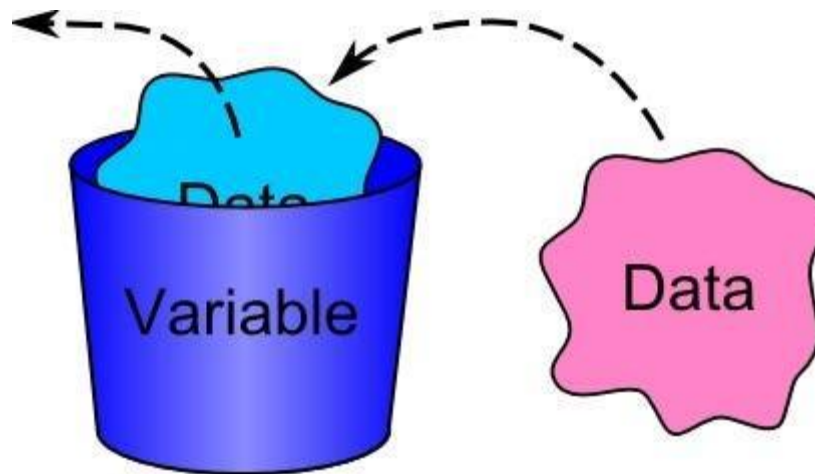
Variables & Memory

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Variables

- A **variable** is a named storage location in memory used to **store** data during program execution.
- It's used to hold data that can **change** during the execution of a program.
- **Think** of it as a **labeled** container for a **value**.



1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

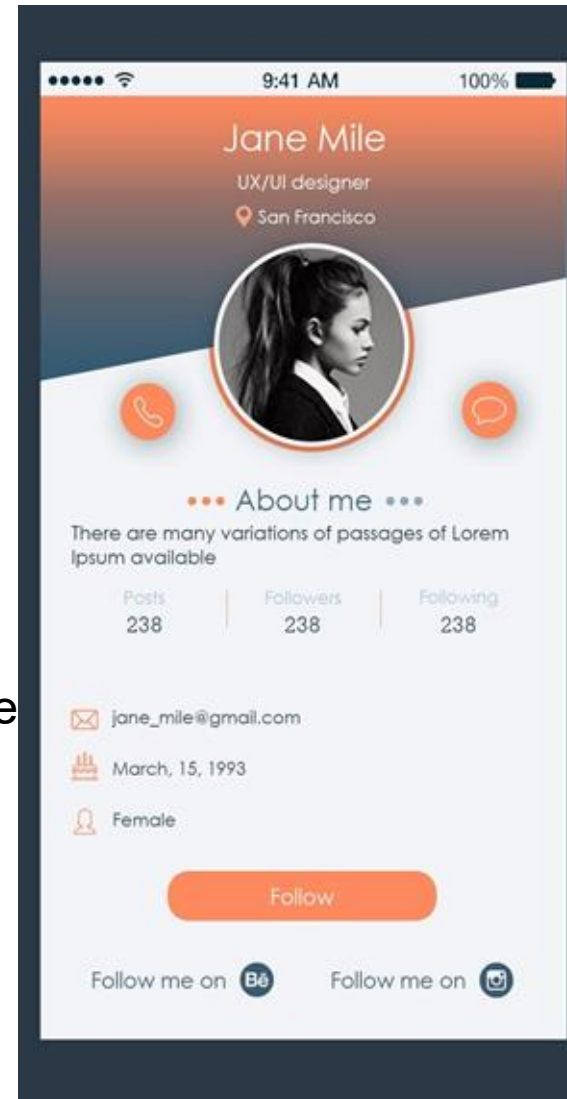
Variables & Memory

Variables

- We **understand** that if an application like **Facebook** needs to **store data** (such as a person's name or age), programmers must **create variables** to **hold** that data within the application.

Example : User Profile

- When you create an account, Facebook stores your **name, age, and email**.
- These pieces of information are stored in *variables* like
Name → "Sita Sharma"
Age → 22
- When you open your profile, Facebook shows your own details by fetching these variable values.



1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Variables

So, How to **CREATE** a Variable in a Program ?

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Variables

Variable Declaration Syntax :

`<dataType> variableName;`

Eg: **int age;**

Variable Initialization Syntax:

`variableName = Value;`

Eg: **age = 16 ;**

Declaration and Initialization at the same line:

`<dataType> variableName = Value;`

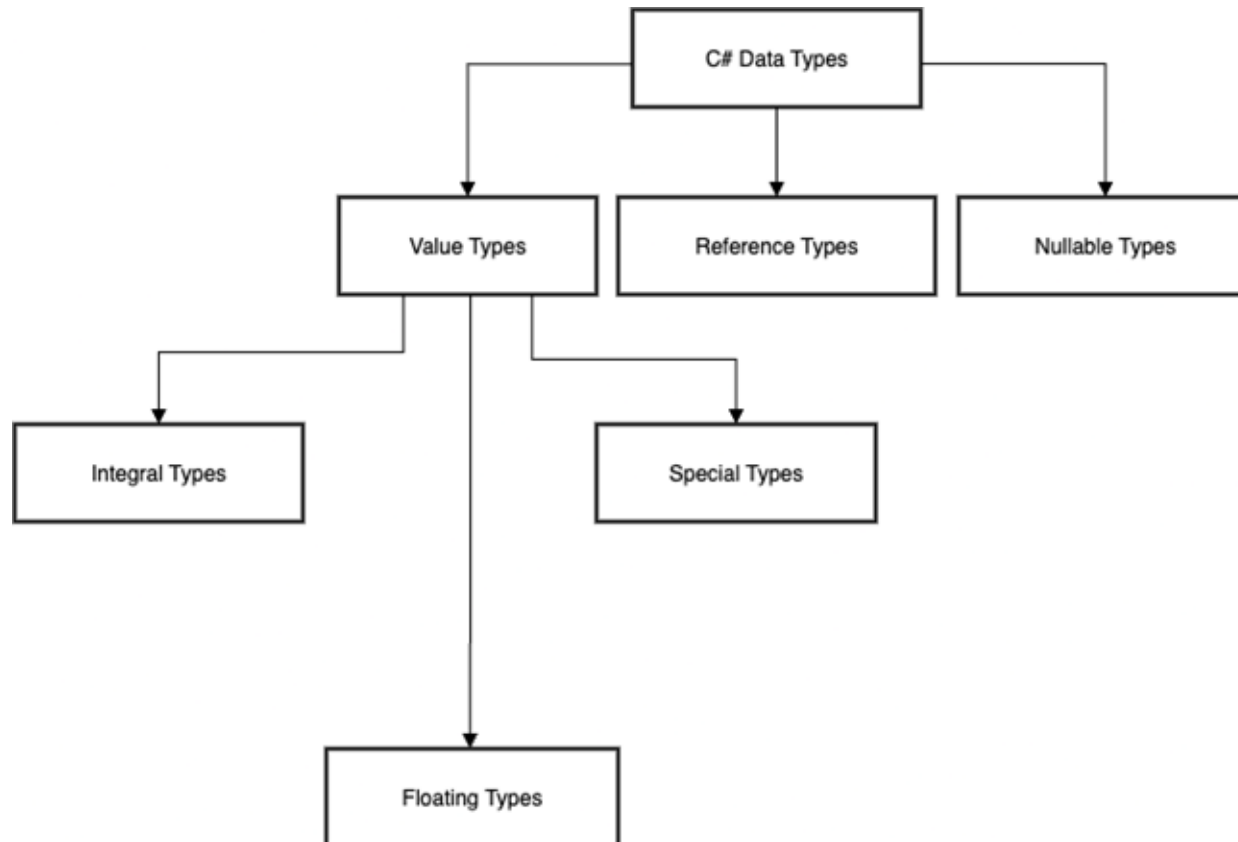
Eg: **int age = 16 ;**

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

DataTypes

A data type specifies the **size and type** of variable values.



1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Data Types - Integral

Value Types

- Store the **actual value** directly in stack memory.
- **Characteristics:**
 - Fast to access (stack-based).
 - Each variable has its own copy of the data.
 - Ideal for small, simple data (numbers, boolean, characters).

Examples: `int users = 100;` variable stores **100** directly in stack.

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Value Types - Integral

Integral Types

Whole number data types.

Use Case: Counting, IDs, loop counters.

Type	Size	Range	Example
sbyte	8-bit	-128 to 127	sbyte temp = -15;
byte	8-bit	0 to 255	byte alpha = 255;
short	16-bit	-32,768 to 32,767	short sensor = 30_000;
ushort	16-bit	0 to 65,535	ushort port = 8_080;
int	32-bit	-2,147,483,648 to 2,147,483,647	int users = 10_00_000;
uint	32-bit	0 to 4,294,967,295	uint id = 4_294_967_295;
long	64-bit	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	long stars = 100_000_000_000;
ulong	64-bit	0 to 18,446,744,073,709,551,615	ulong universe = 18_400_000_000_000_000_000;

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Value Types – Floating

Floating Types

Numbers with **decimal points**.

Use Case: Scientific calculations, precision work.

Type	Size	Example
float	32-bit	float val = 3.14f;
double	64-bit	double val = 3.1415926535;
decimal	128-bit	decimal val = 3.141592653589793238m;

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Value Types – Floating

Quiz: Why does C# provide multiple integral data types such as byte, short, int, and long?

- A. To confuse programmers
- B. To allow programmers to choose a type based on memory requirement and range
- C. To make the language more complex
- D. Because each data type performs different arithmetic operations



- 1. C# Syntax Essentials
 - 1. Comments, Indentation, Statements, Blocks, Expressions
 - 2. Top-Level Statements
 - 3. Identifiers and Naming Conventions
- 2. Variables & Memory
 - 1. Declaring & Initializing Variables
 - 2. const & readonly
 - 3. Built-in Types: Guid, DateTime, TimeSpan
- 3. Strings
 - 1. String literals
 - 2. Interpolated Strings
- 4. Arrays
 - 1. Single-Dimensional Arrays
 - 2. Common Array Methods
- 5. Generic Collections
 - 1. List
 - 2. Dictionary

Variables & Memory

Value Types – Floating

Quiz: Why does C# provide multiple integral data types such as byte, short, int, and long?

- A. To confuse programmers
- B. To allow programmers to choose a type based on memory requirement and range
- C. To make the language more complex
- D. Because each data type performs different arithmetic operations

Correct Answer: B

Reason: Different integral types allow developers to optimize **memory** and **performance** based on how large or small the values are.

- 1. C# Syntax Essentials
 - 1. Comments, Indentation, Statements, Blocks, Expressions
 - 2. Top-Level Statements
 - 3. Identifiers and Naming Conventions
- 2. Variables & Memory
 - 1. Declaring & Initializing Variables
 - 2. const & readonly
 - 3. Built-in Types: Guid, DateTime, TimeSpan
- 3. Strings
 - 1. String literals
 - 2. Interpolated Strings
- 4. Arrays
 - 1. Single-Dimensional Arrays
 - 2. Common Array Methods
- 5. Generic Collections
 - 1. List
 - 2. Dictionary

Variables & Memory

Value Types – Special

Special Types

Special categories like true/false, characters, or enums.

Use Case: Boolean logic, character representation, and predefined value sets for improved code readability.

Type	Size	Example
bool	1-bit	bool val = true;
char	16-bit	char val = 'A';
enum	32-bit(Typically)	enum Days { Sun, Mon, Tue }

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Reference Types

Reference

Store a reference (address) to the data, which is kept in heap memory.

Characteristics:

- More flexible, can hold complex structures.
- Two variables can reference the same object.

Type	Size	Example
string	32 / 64	string name = "John";
object	32 / 64	object config = ...;
class	32 / 64	class User { ... }
array	32 / 64	int[] primes = [2,3,5];
delegate	32 / 64	Func<int> roll = ...;

Use Case: Complex data structures, object-oriented programming, collections.

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Nullable Types

Nullable value types

- Normally, **value types** (like `int`, `float`, `bool`) **cannot hold null**.
- **Nullable value types** allow value types to **optionally store null**, making them useful for optional data.

```
1  int? age = null;           // Nullable int
2  bool? isActive = null;    // Nullable bool
```

- The **?** symbol marks a value type as **nullable**.
- Nullable Value types started in **C# 2.0**

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. `const` & `readonly`
 3. Built-in Types: `Guid`, `DateTime`, `(TimeSpan)`
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Nullable Types

Nullable Reference Types

- Traditionally, **reference types** (like string, object) **can always be null**.
- Nullable reference types** (NRTs) let you **explicitly indicate if a reference can be null**, improving **null safety** at compile time.
- Introduced in **C# 8.0**.
- Enabled via **project settings** or `#nullable enable`.

```
string name = null; // ⚠ Warning: assigning null to non-nullable string
```

```
#nullable enable
```

```
string? name = null; // Nullable reference type  
string title = "Hello"; // Non-nullable reference typed
```

- C# Syntax Essentials
 - Comments, Indentation, Statements, Blocks, Expressions
 - Top-Level Statements
 - Identifiers and Naming Conventions
- Variables & Memory
 - Declaring & Initializing Variables
 - const & readonly
 - Built-in Types: Guid, DateTime, TimeSpan
- Strings
 - String literals
 - Interpolated Strings
- Arrays
 - Single-Dimensional Arrays
 - Common Array Methods
- Generic Collections
 - List
 - Dictionary

Variables & Memory

Nullable Types

Nullable Reference Types : Importance / Use Cases

- **Prevent `NullReferenceException`** by catching null issues **at compile time**.
- **Clear documentation** of which variables can or cannot be null.
- **Better API design** — users know what can be null.

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Const & Read Only

In **C#**, both **const** and **readonly** enforce **immutability**, but they solve fundamentally different problems:

const → Compile-time constant

- Value must be known at compile time
- Change requires recompiling all dependent code
- Used for **fixed, never-changing values**

```
const double Pi = 3.14159;  
const int MaxMarks = 100;
```

Think of it like a "hard-coded" value.



1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Const & Read Only

readonly → Runtime constant

- Value can be set **only once**, but **at runtime** (e.g., inside a constructor)
- **Not baked** into compiled code — safer for updates
- Used for values that are **constant per object**, but not known until runtime

```
public class Program
{
    readonly DateTime CreatedAt;
    0 references
    public Program()
    {
        CreatedAt = DateTime.Now; // allowed
    }
}
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

Guid

Guid – Globally Unique Identifier

- Represents a 128-bit unique identifier.
- Ensures uniqueness across systems and time.
- Commonly used for keys, session IDs, or distributed systems.

```
Guid id = Guid.NewGuid();  
Console.WriteLine(id);  
// Example: 38d977a7-61e5-4f9e-8acd-9f5a9f8d1e4a
```

```
Guid empty = Guid.Empty; // 00000000-0000-0000-0000-000000000000
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

DateTime

DateTime – Date and Time

- Represents a specific point in time, typically expressed as date and time of day.
- Range: 0001-01-01 to 9999-12-31.

```
DateTime now = DateTime.Now;  
DateTime utcNow = DateTime.UtcNow;  
DateTime specific = new DateTime(2025, 4, 5, 10, 30, 0);  
  
Console.WriteLine(now.ToString("yyyy-MM-dd HH:mm"));
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

TimeSpan

TimeSpan – Duration or Interval

- Represents a time interval (e.g., difference between two dates).
- Useful for measuring elapsed time or scheduling.

```
TimeSpan interval = new TimeSpan(2, 30, 0); // 2h 30m  
TimeSpan duration = DateTime.Now - someEarlierTime;
```

```
Console.WriteLine(duration.TotalMinutes);
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

The var Keyword

var

- **var** is a C# keyword used for **implicitly typed local variables**.
- The compiler determines the variable's type **at compile time**.
- Once the type is determined, it cannot be changed.

```
var name = "John";  
var age = 25;
```

- **var** must be **initialized** when declared.

```
var price;
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Variables & Memory

The Dynamic Keyword

Dynamic Typing

- Normally, **variables** (like int, string, bool) **are type-checked**.
- **dynamic variables** skip that, instead checking happens **at runtime** and the type can even change during execution.

```
dynamic value = 10;    // int, for now  
value = "Hello, C#!"; // now a string – no error
```

Why dynamic keyword?

- Using this keyword, the variable can hold different types, giving more flexibility.
- However, errors may only appear when the program runs.

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Strings

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Strings

Introduction

What is a string?

- A string is a built-in data type in **C#** used to represent text.
- It is an immutable reference type that stores a sequence of Unicode characters (char).
- Declared using the string keyword (alias for System.String).
- **Example: "Bikram", "Bishal", "Apple"**

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, **TimeSpan**
- 3. Strings**
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Strings

Introduction

Common Use Case

- Storing names, addresses, messages
- File paths, URLs
- User input/output
- Data formatting and display

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, **TimeSpan**
3. **Strings**
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Strings

Types – String Literals

String Literals

- Can be declared with string keyword or @ (verbatim) for raw strings.
- Represent text data using double quotes.

```
string name = "Hello World";  
string path = @"C:\Users\John\Documents"; // Verbatim string (no escape needed)  
string multiline = "Line 1\nLine 2"; // Escape sequences work
```

- ✓ Use @ to avoid escaping backslashes
- ✓ Supports escape sequences: \n, \t, \", etc.

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Strings

Types – Interpolated Strings

Interpolated Strings

- Allow embedding expressions directly in strings using \$.
- Syntax: \$"{expression}"

```
string name = "Alice";  
int age = 30;
```

```
string message = $"Hello, {name}! You are {age} years old.";  
Console.WriteLine(message);  
// Output: Hello, Alice! You are 30 years old.
```

Can include:

- Variables: {name}
- Expressions: {age * 2}
- Methods: {DateTime.Now.ToString("yyyy")}

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Introduction to C# & .NET

Arrays

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Arrays

Types - Single-Dimensional Arrays

Single-Dimensional Arrays

- Stores elements in a linear sequence accessible by a single index.
- Most commonly used array type in C#.

Declaration & Initialization:

```
// 1. Declare and assign later
int[] numbers = new int[5]; // Array of 5 integers (default: 0)

// 2. Initialize with values
string[] names = { "Alice", "Bob", "Charlie" };

// 3. Using 'new' with explicit size
double[] prices = new double[3] { 10.99, 20.50, 30.00 };
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Arrays

Types - Single-Dimensional Arrays

A **single-dimension array** is a list of items stored in a single row, where each item can be accessed using its position number (called an index).

In simple words: it's like a row of boxes, and each box holds one value that you can reach by its number.

**Accessing
elements:**

```
Console.WriteLine(names[0]); // Output: Alice  
numbers[2] = 42; // Set value at index 2
```

Iterating through array:

```
// Using for loop  
for (int i = 0; i < names.Length; i++)  
{  
    Console.WriteLine(names[i]);  
}  
  
// Using foreach  
foreach (string name in names)  
{  
    Console.WriteLine(name);  
}
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Arrays

Common Methods

C# provides powerful static methods in the Array class to work with

Method	Purpose	Example
Array.Sort()	Sorts elements in ascending order	Array.Sort(numbers)
Array.Reverse()	Reverses the order of elements	Array.Reverse(numbers)
Array.IndexOf()	Returns index of first occurrence, or -1 if not found.	int pos = Array.IndexOf(names, "Bob");
Array.Find()	Finds first element matching a condition (uses lambda)	string match = Array.Find(names, n => n.StartsWith("A"));
Array.Clear()	Sets a range of elements to default value	Array.Clear(numbers, 0, 2);

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Generic Collections

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Generic Collections

Introduction

What Are Generic Collections?

- Part of **System.Collections.Generic**
- Type- safe collections that work with any data type, specified at declaration.
- Eliminate the need for casting and reduce runtime errors.
- **Example: List<String>**

Key Benefits:

Type Safety: Compile-time checking while inserting data

Reusability: Same logic for int, string, custom objects, etc.

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Generic Collections

Common Generic Collections

Type	Purpose
List<T>	Dynamic array – add/remove items freely.
Dictionary<Tkey, Tvalue>	Reverses the order of elements.
Queue<T>	FIFO (First-In, First-Out) structure.
Stack<T>	LIFO (Last-In, First-Out) structure.

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Generic Collections

Types – List<T>

List<T> Overview

- A **resizable, ordered** collection of elements of type **T**
- Similar to arrays, but can **grow or shrink dynamically**
- Found in **System.Collections.Generic**

```
List<int> numbers = new(); // Empty list
List<string> names = new() { "Alice", "Bob" }; // With initial values
List<double> prices = new List<double>(); // Alternative syntax
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Generic Collections

Common Methods – List<T>

Common methods of List<T>

Type	Purpose
.Add(item)	Adds an item to the end.
.Remove(item)	Removes first occurrence.
.RemoveAt(index)	Removes item at index.
.Count	Returns number of elements.
.Contains(item)	Checks if item exists.
.Clear()	Removes all items.

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Generic Collections

Example Usage – List<T>

```
List<string> fruits = new();  
fruits.Add("Apple");  
fruits.Add("Banana");  
fruits.Remove("Apple");  
  
foreach (string fruit in fruits)  
{  
    Console.WriteLine(fruit);  
}  
  
// Output: Banana
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Generic Collections

Dictionary<TKey, TValue> – Key-Value Storage

Dictionary<TKey, TValue> Overview

- Stores data as **key-value pairs**
- Enables **fast retrieval** using a unique key
- Keys must be unique and not null

```
Dictionary<int, string> employeeNames = new();  
Dictionary<string, double> prices = new();  
  
// With initial data  
var settings = new Dictionary<string, object>  
{  
    { "Volume", 80 },  
    { "Fullscreen", true },  
    { "Brightness", 75.5 }  
};
```

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Generic Collections

Common Methods - Dictionary<TKey, TValue>

Type	Purpose
.Add(key, value)	Inserts a new key-value pair.
.Remove(key)	Removes entry by key.
.ContainsKey(key)	Check if key exists.
.TryGetValue(key,out value)	Safety get value without exceptions.
.Count	Number of key-value pairs

1. C# Syntax Essentials
 1. Comments, Indentation, Statements, Blocks, Expressions
 2. Top-Level Statements
 3. Identifiers and Naming Conventions
2. Variables & Memory
 1. Declaring & Initializing Variables
 2. const & readonly
 3. Built-in Types: Guid, DateTime, TimeSpan
3. Strings
 1. String literals
 2. Interpolated Strings
4. Arrays
 1. Single-Dimensional Arrays
 2. Common Array Methods
5. Generic Collections
 1. List
 2. Dictionary

Any Questions?